Project Title: Ski Resort Gear Rental System

Course: CS340
Team 80 Members: Joshua Cabalka, Jack Boland, Philip Gadsden
1/14/2026

Feedback Received:

Skyla Chen 2d

Hello Team 80!

- **Your review should provide helpful and insightful answers to the following questions**:
  - Does the overview describe what problem is to be solved by a website with DB back end? If yes, summarize. If not, what changes would better support describing the problem to be solved?
    - I think the overview does a great job with describing the problem, with how the system manages ski equipment, rentals, and maintenance for staff.
  - Does the overview list specific facts? If yes, summarize what the facts illustrate about the proposed DB solution. If not, what facts would better support illustrating the scope and scale of the proposed DB solution?
    - I'd say that there could be a bit more on specific facts, as the overview mainly describes the capabilities of the management system, rather than facts with manageable numbers.
  - Are at least four entities described, and does each one represent a single idea to be stored as a list? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?
    - Yes there are at least four entities described, as there are 5 provided. They do a great job of representing their own concept.
  - Does the outline of entity details describe the purpose of each, list attribute datatypes and constraints, and describe relationships between entities? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?
    - Yeah, I think that the outline of the entity details describe the purpose, datatypes, constraints and relationships pretty well as they are all written in the document. Great Job!
  - Are 1:M relationships correctly formulated? Is there at least one M:M relationship? Does the ERD present a logical view of the database? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?
    - Yes, the 1:M looks good:
      - customers →rental_orders
      - employees →rental_orders
      - employees→service_tickets.
    - As well as the M:M relationships:
      - gear_items<->rental_orders
      - gear_items<->service_tickets
  - Is there consistency in a) naming between overview and entity/attributes b) entities plural, attributes singular c) use of capitalization for naming? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?
    - Reading over the document, naming is pretty much consistent! Amazing job team 80!

All original work, besides the questions which were provided in the review assignment.

D David Hausman 2d

The system is designed to manage a ski resort rental system at the item level. Specifically it tracks: Individual gear, availability and condition, rental transactions, checkout/return history, maintenance and repair.

Each piece of gear is individually tracked. Orders are capable of handling multiple gear items. Time stamps to create a temporal record of checkout/return. Service work is implemented independently.

1.Customers

2.Employees

3.Gear Items

4.Rental Orders

5.Service Tickets

With each entity representing a distinct atomic concept as outlined in the course requirements(as well as being appropriate for a well designed database) this design avoids unnecessary mixing of concerns.

For each entity described in the outline the document provides: The purpose(implicit through strict naming semantics), datatypes, constraints, and relationships are described.

1:M relationships appear to be correctly formulated and are in sufficient numbers to meet assignment requirements. Same for the M:M relationship.

The ERD appears to present an accurate representation of the textual description.

Names appear to be correct and consistent.

Looks good to me!

I made this.

S Sammy Alyooi 2d

I think group 80's overview addresses the web portal and database for the Ski Resort Gear Rental System in good detail. They clearly stated the purpose of their system as an admin-facing ski resort system focused on managing equipment inventory, rentals, and maintenance. I think the fact that they address the responsibilities of their system upfront will help them with planning and design.

They address several facts about their system and the approach they are using. For example, they highlight in detail what types of transactions will take place in the ski resort and how their frontend and database will handle these activities: "Rental activity is recorded through orders created by employees for customers.". They also expand on other operations for their system, such as keeping a record of historical data or tracking repairs and maintenance separately, which is something that can fit an object-oriented design.

There are six main entities and one bridge entity, rental_order_items. Each entity, such as customers, employees, or rental_orders, represents a single idea or responsibility that can be performed by that entity. As expected in business and system models like this, many entities have 1:M relationships, with rental_orders and gear_items having M:N relationship via rental_order_items. The 1:M relationships are formulated to mimic real-life situations. For example, customers have a 1:M relationship with rental_orders, and service_tickets have 1:M relationship with service_ticket_items, with the use of foreign keys such as the rental_orders attribute created_by_employees being used as a foreign key referencing employee(employee_id).

I think there is consistency in naming entities and attributes, although there is some naming overlap, such as rental_orders, rental_order_items, service_tickets, and service_ticket_items. I think it could be better if there were a bit more distinction between these names.

All the above analysis is mine.

Arthur Bikele 3d

The overview clearly explains the purpose of the system and the problem it is trying to solve. It shows that the ski resort needs a structured way to manage gear inventory, rental transactions, and maintenance history. By emphasizing tracking individual gear items and their availability, condition, and service history, the overview makes it clear why a database-backed system is necessary. It effectively describes how the system supports daily operations and long-term record keeping.

The overview also includes specific facts that help define the scope of the project. For example, it explains that gear items are tracked individually, that orders can contain multiple items, and that maintenance is handled through service tickets with timestamps. These details show that the database must support transactional data, historical tracking, and inventory management, which gives a good sense of the scale and complexity of the system.

There are more than four entities described, and each one represents a clear and separate concept that should be stored as a list. The entities include customers, employees, gear_items, rental_orders, and service_tickets, along with two intersection tables. Each entity focuses on one idea: customers represent renters, employees represent staff, gear_items represent physical equipment, rental_orders represent transactions, and service_tickets represent maintenance work. This fits well with the object and transaction table patterns discussed in class.

The outline of entity details is strong. Each entity includes its purpose, attributes, datatypes, constraints, and primary keys. Relationships are also clearly described using foreign keys. One issue I noticed is in the rental_orders table: customer_id: INT, NN, FK > employees(employee_id) This should reference the customers table instead of the employees table. It should be: FK > customers(customer_id) Besides that, the entity definitions are clear and well-structured.

The relationships are mostly formulated correctly. Customers have a 1:M relationship with rental_orders, employees have 1:M relationships with both rental_orders and service_tickets, and gear_items has M:M relationships with both rental_orders and service_tickets. These M:M relationships are properly resolved using the rental_order_items and service_ticket_items intersection tables. The ERD logically represents how the system operates and supports both rentals and maintenance tracking.

Naming conventions are consistent and easy to follow. Entities are plural (customers, employees, gear_items, rental_orders), attributes are singular, and snake_case is used consistently. The naming matches between the overview and the database outline, which makes the design easier to understand. Datatypes and constraints are also applied thoughtfully, especially with ENUMs for status and condition fields.

Overall, this draft is very strong and feels complete. The system design is realistic, the relationships are well thought out, and the database structure aligns well with course concepts. Fixing the one foreign key reference in rental_orders would make the design fully consistent.

This review is entirely my own original work.

James Anderson 4d

This system is designed to keep track of gear rentals to customers and maintenance to gear items. The system will track individual item rental history and maintenance history. There are five entities described.

First is the customers entity which records customers who rent gear from the ski resort. The customers entity includes an id which is the primary key, full name and optional phone and email address. This has a one to many relationship with the rental_orders entity.

The second entity described is the employee entity. This table records employees who create rental orders and service tickets. It includes an employee id which is the primary key, a full name, the employee role and, this is where I am a little fuzzy, an is_active attribute. I think this may be if the employee is still employed or on shift? This entity has one to many relationships with the rental_orders and service_tickets entities.

The next entity described is gear_items. This table records each physical piece of ski or snowboard related equipment. The primary key is the gear id. Other attributes include a category which is the type of gear, brand, model, serial number, size, condition, status which describes whether the piece is rented, being serviced, available or retired, and a date which the piece was acquired by the company. The gear_items entity has many to many relationships with rental_orders and service_tickets making proper use of the required junction tables.

The fourth entity is the rental_orders table. This table records each rental checkout transaction. This table shows a rental_id which is the primary key, customer_id, a created_by_employee_id and a created_at which is a date/time. There is an issue here where the outline has the customer_id pointing to the employee entity though it should point to the customer entity. Your ERD is set up correctly but the outline points to a different entity. The rental_orders entity has one to many relationships with customers and employees and a many to many relationship with gear_items.

The fifth and final entity table required for a three person group is the service_tickets entity.This records the maintenance and repair work for gear items. This table includes attributes which are service_ticket_id which is the primary key, opened_by_employee_id, a gear status and a created_at attribute for the time and date for creation. This entity has a one to many relationship with employees and a many to many relationship with gear_items.

There are two intersections tables which are required for a 3 person group. With the exception of one or two minor attribute names I would say that each entity and it's attributes are named appropriately with names that accurately describe their intended use.

The ERD does present a logical view of the database and is easy to follow. In my opinion this draft looks clean and complete.

This review is my own original work.

Jennifer Ho 2d

Hi group 80!

The overview was very straightforward and clearly laid out what functions the system will provide and how the database will help support them. Since the system will not only have to track the availability of rental gear but also any service tickets and the gear's condition, it becomes clear why a database would be needed since several things need to be tracked at once in order to avoid availability and inventory issues.

One thing that your draft is lacking, however, are some facts to help illustrate the scale of the database. For example, I think you could have listed numbers related to inventory, number of employees, and number of customers. This would help your audience get a better idea of the scale of operations and further support the need for a service with a DB backend.

Your database outline does include the required number of entities and a couple M:M relationships are established through the intersection tables. The attributes for each entity also seem to meet the required info for handling the operational processes described in the overview by recording things like the gear check out and return by date. It's already been pointed out by a couple other people, so I won't go into detail, but there is an issue with the customer_id attribute in the rental_orders table where it uses the employee_id as a FK. Aside from that, I don't see any other issues with your 1:M relationships.

Lastly, the only other issue I saw was a naming inconsistency in the rental_order_items table. It takes foreign keys from rental_orders and gear_items, but the key from the former is named "rental_order_id" instead of just "rental_id". Overall though, your draft is looking great with just a few issues that can be resolved fairly quickly.

All the above is my own work.

Summary of Changes:

1. Added numerical scope facts to the overview (inventory size, typical rental volume and service ticket volume) to better illustrate scale of operations.
2. Fixed foreign key reference so *rental_orders.customer_id* points to *customers(customer_id)*.
3. Added a comment to *employees.is_active* to describe whether an employee is currently employed and is authorized to create orders and service tickets.

Overview

This project models an admin-facing ski resort gear rental system designed to manage equipment inventory of around 300 individual gear items across various categories, 80 rental transactions per week and 20 gear maintenance service tickets per week. The system will track gear items, which will allow staff to monitor availability, condition, rental and service history at an item level. Rental activity is recorded through orders created by employees for customers, with each order being capable of having multiple gear items. A typical rental order contains 2-4 gear items. Historical data will be kept with timestamps when items were checked out and returned. Maintenance and repair work will be tracked separately with service tickets that will document service procedures performed on one or more gear items along with timestamps for work start and completion.

Database Outline

1) *customers:* Records customers who rent gear from the ski resort.

- customer_id: INT, AI, PK, NN
- first_name: VARCHAR(100), NN
- last_name: VARCHAR(100), NN
- phone: VARCHAR(20), NULL
- email: VARCHAR(255), NULL

Relationship:

- 1:M relationship with rental_orders

2) *employees:* Records employees who create rental orders and service tickets.
   - employee_id: INT, AI, PK, NN
   - first_name: VARCHAR(100), NN
   - last_name: VARCHAR(100), NN
   - role: VARCHAR(50), NN
   - is_active:BOOLEAN, NN, DEFAULT TRUE
     - “is_active indicates if the employee is currently employed and can create orders/tickets”

   Relationships:
   - 1:M relationship with rental_orders
   - 1:M relationship with service_tickets

3) *gear_items:* Records each physical piece of ski/snowboard related equipment
   - gear_item_id: INT, AI, PK, NN
   - category:ENUM(‘snowboard’, ‘boots’, ‘helmet’, ‘other’), NN
   - brand: VARCHAR(100), NN
   - model: VARCHAR(100), NN
   - serial_number: VARCHAR(100), UQ, NN
   - size: VARCHAR(20), NN
   - condition_grade: ENUM(‘new’, ‘good’, ‘fair’, ‘needs_repair’), NN
   - status: ENUM(‘available’, ‘rented’, ‘in_service’, ‘retired’), NN
   - acquired_at: DATETIME, NN

   Relationships:
   - M:M relationship with rental_orders
   - M:M relationship with service_tickets

4) *rental_orders:* Records each rental checkout transaction
   - rental_order_id: INT, AI, PK, NN
   - customer_id: INT, NN, FK > customers(customer_id)
   - created_by_employee_id: INT, NN, FK > employees(employee_id)
   - created_at: DATETIME, NN

   Relationships:
   - 1:M relationship with customers
   - 1:M relationship with employees
   - M:M relationship with gear_items

5) *service_tickets:* Records maintenance and repair work for gear items
    - service_ticket_id: INT, AI, PK, NN
    - opened_by_employee_id: INT, NN, FK > employees(employee_id)
    - status: ENUM('open', 'in_progress', 'completed', 'canceled'), NN
    - created_at: DATETIME, NN

    Relationships:
    - 1:M relationship with employees
    - M:M relationship with gear_items

Intersection Tables

1) *rental_order_items:* Implements the M:M relationship between *rental_orders* and *gear_items*
    - rental_order_id: INT, NN, FK → rental_orders(rental_order_id)
    - gear_item_id: INT, NN, FK → gear_items(gear_item_id)
    - checked_out_at: DATETIME, NN
    - due_at: DATETIME, NN
    - returned_at: DATETIME, NULL

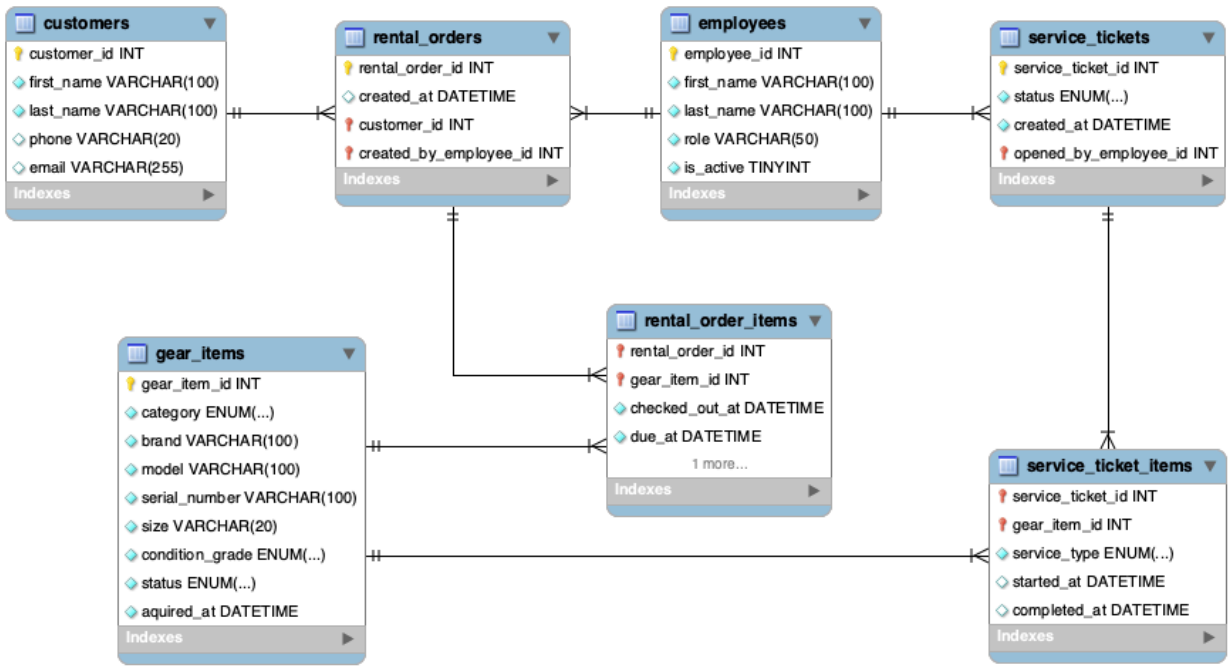
    Primary Key:
    - (rental_order_id, gear_item_id)

    Business rule:
    - A gear item can only appear in one active rental item at a time. This will be enforced by logic using *gear_items.status = 'available'* before checkout and updating status to *rented* while *returned_at* is NULL.

2) *service_ticket_items:* Implements the M:M relationship between *service_tickets* and *gear_items*.
    - service_ticket_id: INT, NN, FK → service_tickets(service_ticket_id)
    - gear_item_id: INT, NN, FK → gear_items(gear_item_id)
    - service_type: ENUM('wax', 'tune', 'repair', 'binding_adjust', 'inspect'), NN
    - started_at: DATETIME, NULL
    - completed_at: DATETIME, NULL

    Primary Key:
    - (service_ticket_id, gear_item_id)

Entity-Relationship Diagram



Citations:
ERD generated with MySQL workbench. Original work by the project team.